Esta clase va a ser

- grabada

CODERHOUSE

DATA SCIENCE

Mejora de modelos de Machine Learning I

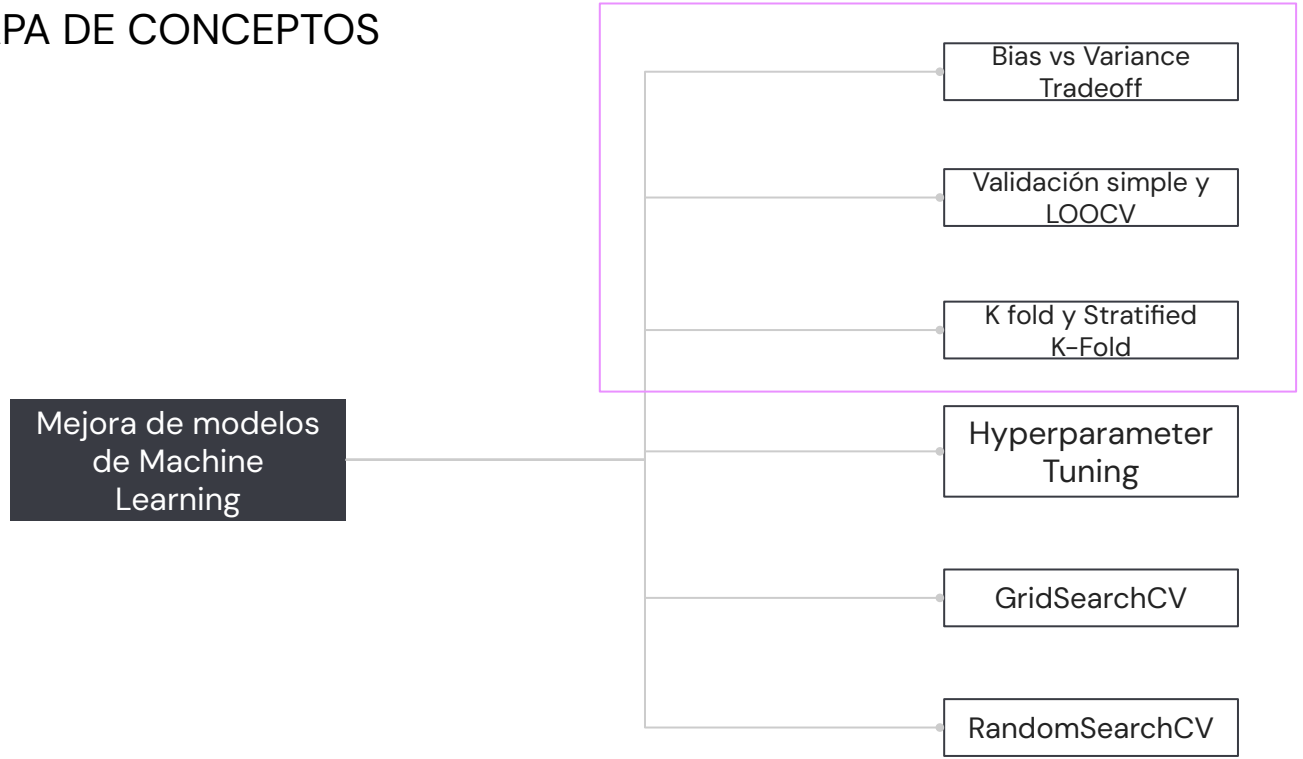
CODERHOUSE

Objetivos de la clase

- **Bias vs. Variance Tradeoff**
- **Validación de modelos**
- **Identificar** conceptos asociados a la mejora de modelos de Machine Learning

CODERHOUSE

MAPA DE CONCEPTOS



CODERHOUSE

Bias vs Variance Tradeoff

CODERHOUSE

Definición

CODERHOUSE

Bias

Es una forma de cuantificar el error de un modelo y se define como la **diferencia entre el valor esperado del estimador (es decir, la predicción media del modelo) y el valor real.**

$$Bias(\hat{\theta}) = E(\hat{\theta}) - \theta$$



CODERHOUSE

Bias

Cuando se dice que un modelo tiene un bias muy alto quiere decir que **el modelo es muy simple y no se ha ajustado a los datos de entrenamiento (suele ser underfitting)**, por lo que produce un error alto en todas las muestras: entrenamiento, validación y test.

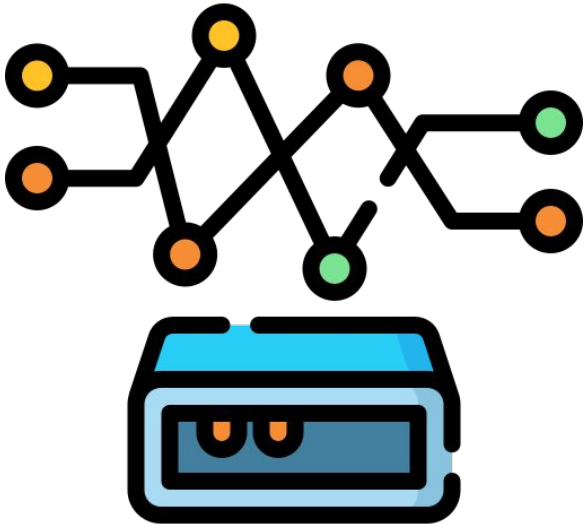


CODERHOUSE

Varianza



Es una medida de dispersión pero se utiliza como **estimador de cuánto varía la predicción según los datos que utilicemos para el entrenamiento**.



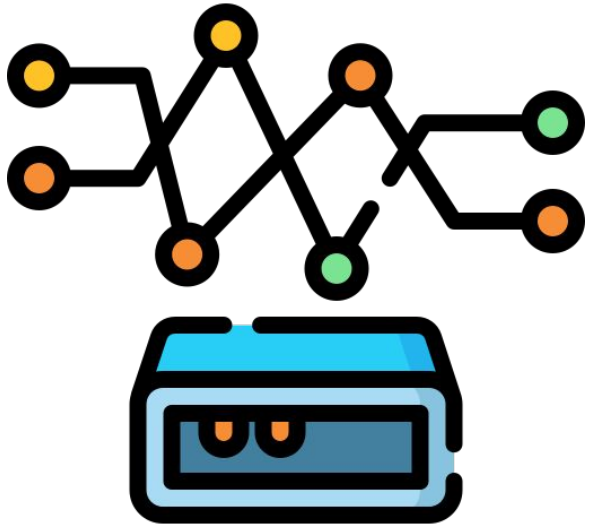
$$\sigma^2 = \frac{\sum (xi - \bar{x})^2}{N}$$

CODERHOUSE

Varianza



La mayoría de los algoritmos de Machine Learning aprenden de los datos de entrenamiento de manera aleatoria. Así que es normal que todos los modelos tengan cierta varianza. Aunque **si creamos un modelo robusto, este debería aprender las relaciones entre las variables y el target**.

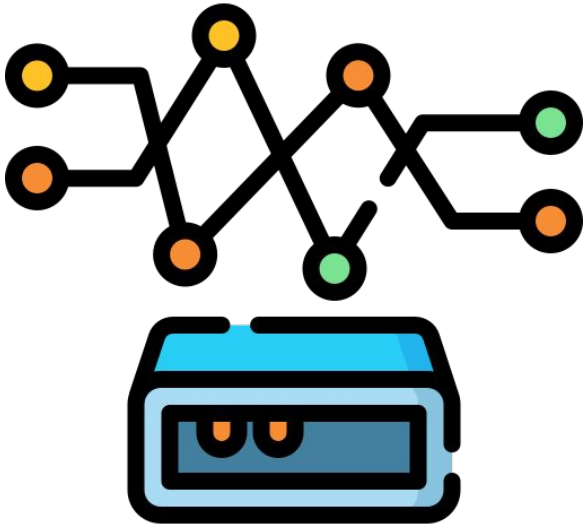


CODERHOUSE

Varianza

- ✔ Un modelo con **varianza baja** indica que cambiar los datos de entrenamiento se producen cambios pequeños en la estimación.
- ✔ Al contrario, un modelo con **varianza alta** quiere decir que pequeños cambios en el dataset conlleva a grandes cambios en el output **(generalmente overfitting)**.





CODERHOUSE

PARA RECORDAR

Bias-Variance Tradeoff

Los algoritmos que suelen tener un error de **Bias (Sesgo) alto** suelen tener una **varianza baja.** Por el contrario modelos con **Bias (Sesgo) alto** suelen tener varianza alta





CODERHOUSE

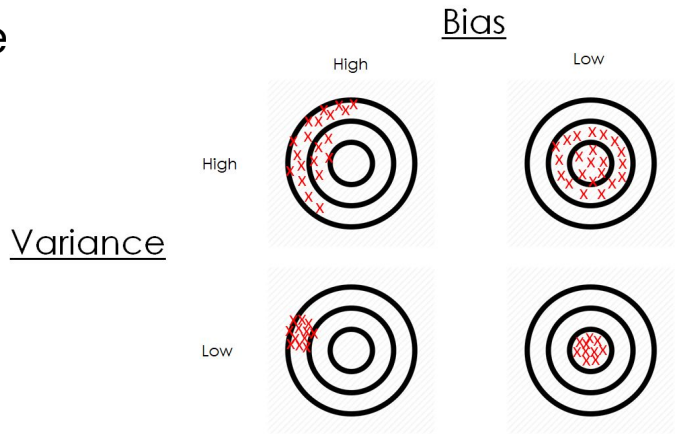
Bias-Variance Tradeoff

CODERHOUSE

Bias-Variance Tradeoff



Es una consideración de diseño al entrenar modelos de aprendizaje automático. Se puede ver como un paradigma a la hora de aplicar modelos de Machine Learning



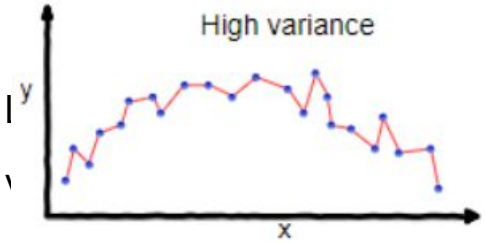
Lo óptimo que queremos tener, es un modelo que tenga poco bias y poca varianza. Esto es lo que podríamos llamar un **modelo robusto**.

CODERHOUSE

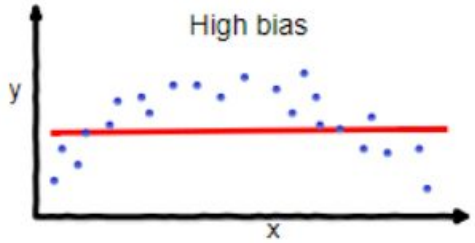
Bias-Variance Tradeoff



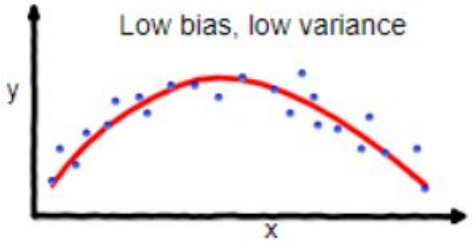
El problema justamente lo tenemos, cuando **no contamos con un modelo robusto y queremos mejorar uno de estos errores.** Aquí se encuentra el famoso *"tradeoff"*.



overfitting



underfitting



Good balance

CODERHOUSE

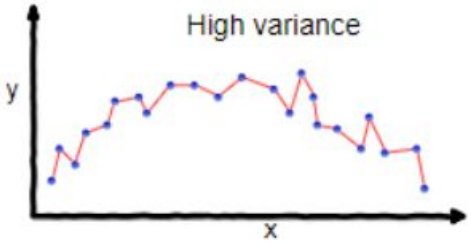
Bias-Variance Tradeoff



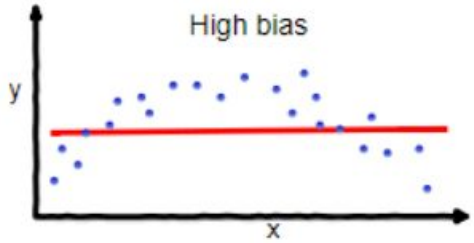
El *"tradeoff"* se explica porque:

- Disminuir la varianza implica aumentar el bias.
- Disminuir el bias hace que la varianza crezca.

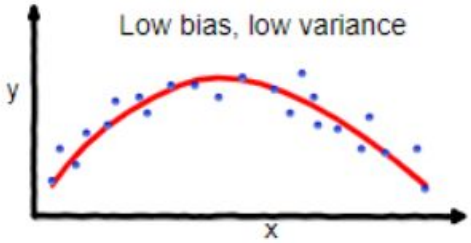
Esto se explica porque un modelo con bias es un **modelo muy simple** y un modelo con varianza es un **modelo muy complejo**.



overfitting



underfitting



Good balance

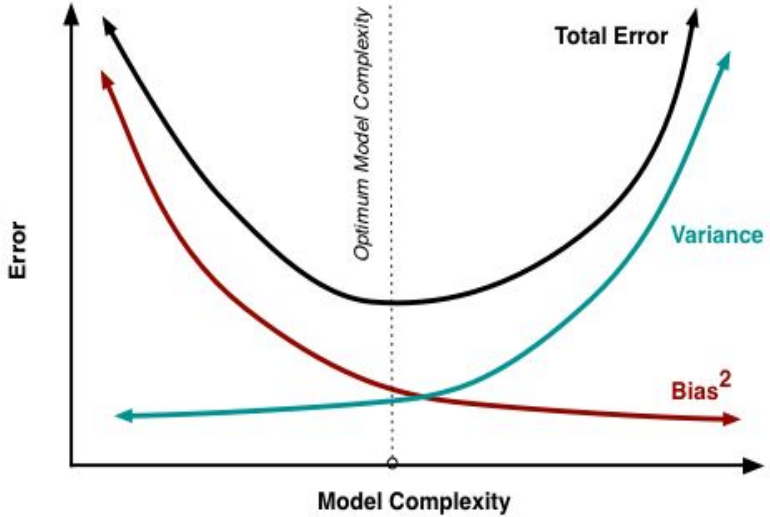
CODERHOUSE

Bias-Variance Tradeoff



Lo que debemos hacer es encontrar un modelo predictivo que tenga un **buen balance entre bias y varianza**.

Esto implica que el modelo no debe ser muy complejo ni muy simple con el fin de minimizar el error en las predicciones:



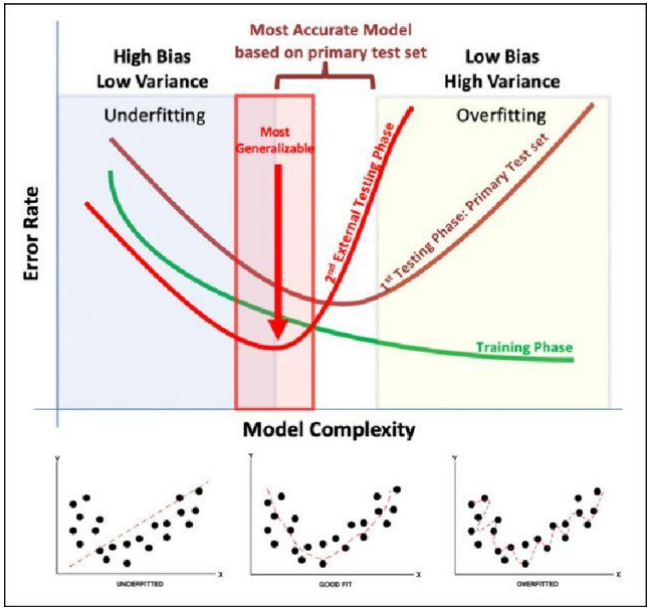
Error= Bias^2+ Varianza + Error Irreducible

CODERHOUSE

Bias-Variance Tradeoff



Por tanto a el punto óptimo será aquel donde se minimice el error total, esto implica que el **error total se minimice** y la complejidad del modelo sea la suficiente para no permitir ya sea **underfitting u overfitting**. Con esto garantizamos que el modelo sea lo más **generalista** posible y no particular



CODERHOUSE

Causas de overfitting

CODERHOUSE

Causas del overfitting



Las causas del overfitting pueden ser diversas complicadas. Sin embargo se tienen tres principales:

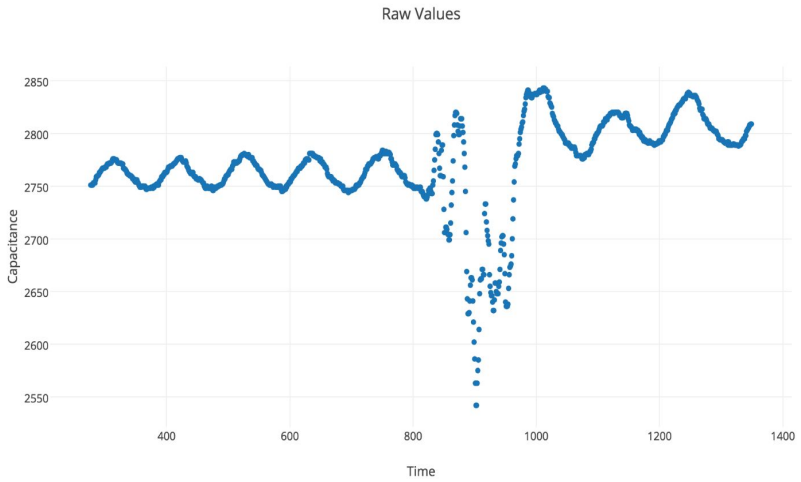
1. Ruido en el conjunto de entrenamiento
2. Mala gestión del Bias-Variance Tradeoff
3. Falta de Datos

CODERHOUSE

1) Ruido en el conjunto de entrenamiento



Cuando el conjunto de entrenamiento es demasiado pequeño o tiene menos datos representativos o demasiado ruido. Esta situación hace que **los "ruidos" tengan grandes posibilidades de ser aprendidos, y luego actuar como base de predicciones.** Por lo tanto, un algoritmo que funcione bien, debería poder distinguir los datos representativos de los ruidos.



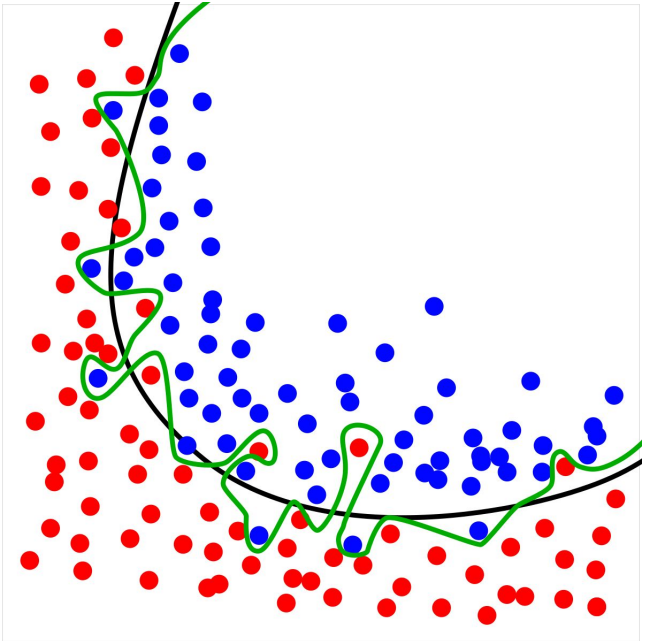
CODERHOUSE

2) Mala gestión del Bias-Variance Tradeoff



Esto se debe a lo visto anteriormente, cuando no se logra gestionar efectivamente el equilibrio entre sesgo y varianza nuestro modelo incluso, puede verse afectado por el underfitting.

El caso opuesto es cuando tenemos un modelo que solo funciona en los datos de entrenamiento (overfitting)

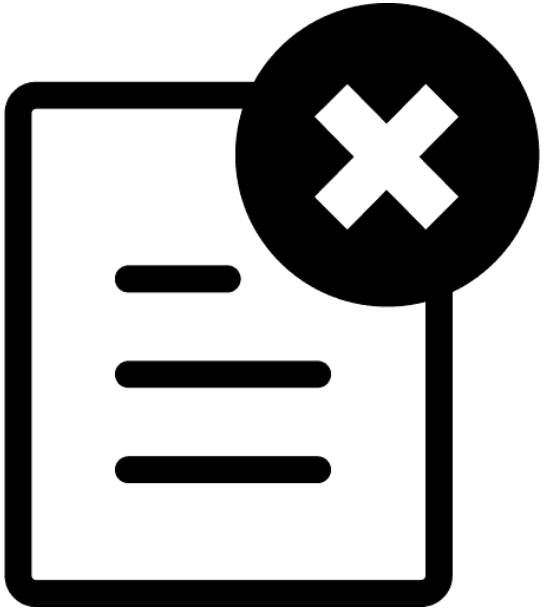


CODERHOUSE

3) Falta de datos



Sin dudas, cuando tenemos pocos registros en nuestro dataset es un problema a la hora de lograr una generalización de nuestro modelo.



Los datos faltantes no permiten que se pueden realizar operaciones matriciales lo que dificulta las operaciones matemáticas desarrolladas por los modelos

CODERHOUSE

¿Cómo evitar el overfitting?

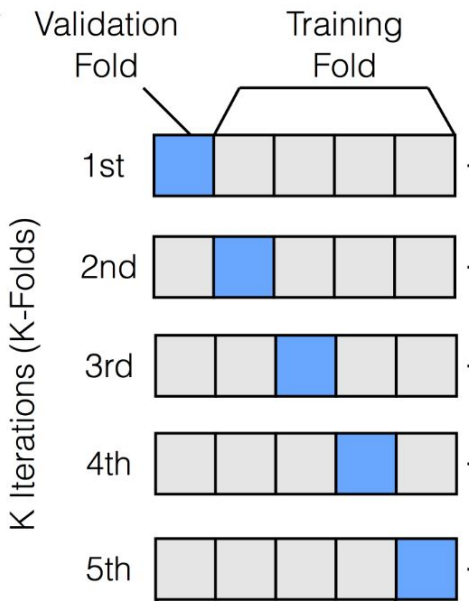
CODERHOUSE

1) Cross Validation



Es una técnica potente para evitar el overfitting. Se utiliza el dataset de train original para generar múltiples datasets y en cada uno ajustamos al modelo.

Existen diversos métodos para aplicar está técnica como por ejemplo K-Fold y LOOCV donde se pueden ajustar los hiperparametros del modelo

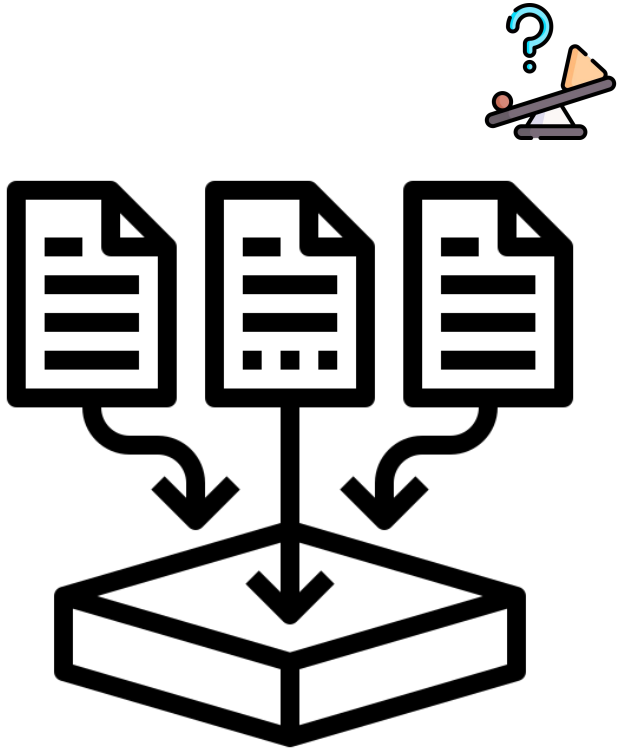


CODERHOUSE

2) Entrenar con más datos

Esto permite que los algoritmos detectan la señal mejor. De igual forma con más datos se previene el overfitting porque aumenta la capacidad de generalización de los modelos.

Sin embargo esta solución puede ser costosa y en algunos casos si la data que se agrega es ruido genera problemas de interpretación



CODERHOUSE

3) Data Augmentation



Es una alternativa cuando no se tienen suficientes recursos para conseguir más datos.

Se puede hacer que los datos que se tienen sea más diversos de tal manera que el se entrenen algoritmos con datasets ligeramente diferentes generando mayor capacidad de discernimiento al modelo.



CODERHOUSE

4) Reducir la complejidad



Se disminuye la complejidad del modelo con el fin de prevenir el overfitting. Esto se puede hacer generando algoritmos que permitan elegir variables apropiadas y que permitan que el número de parámetros no sea tan alto, de esta forma se tienen modelos menos pesados y que se ejecutan más rápido.



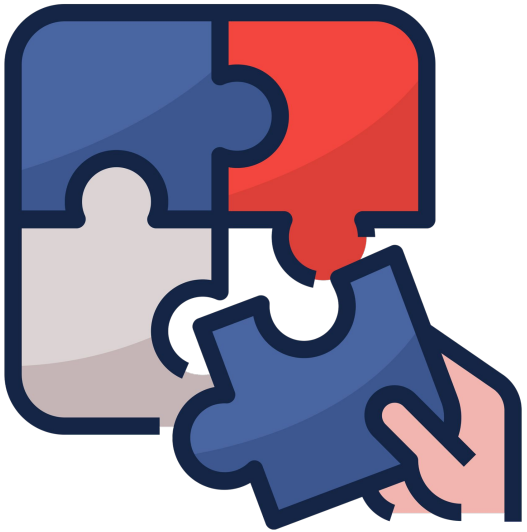
CODERHOUSE

5) Regularización



Conjunto de técnicas que permiten forzar al modelo a ser más simple. Esto se puede hacer utilizando funciones de costo que penalicen por mayor cantidad de parámetros.

La regularización también permite realizar hypertuning de parámetros con el fin de tener modelos más robustos

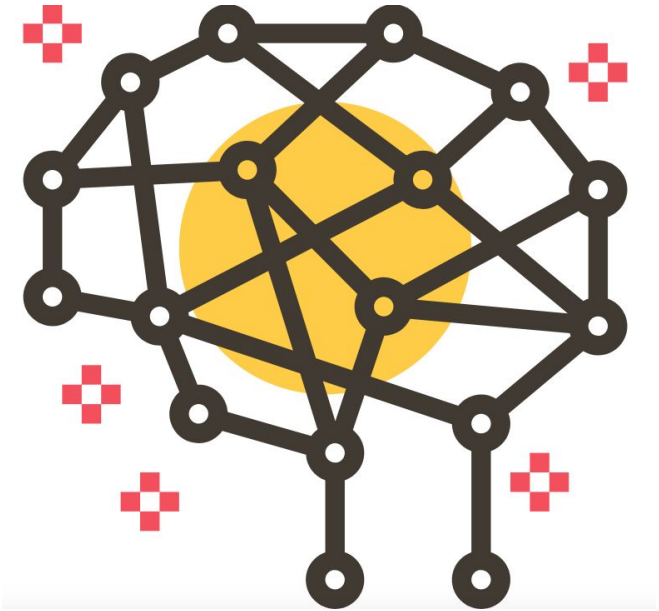


CODERHOUSE

6) Uso de métodos de Ensemble



Combinar predictores de múltiples modelos con el fin de disminuir el error total de los modelos. Existen dos formas distintas (Bagging y Boosting)



Bagging genera el entrenamiento de de modelos en paralelo para optimizar las predicciones

Boosting consisten en usar modelos base e ir incrementando la complejidad

CODERHOUSE



Para pensar

Si tenemos un modelo con alto sesgo, ¿estamos en un escenario de Underfitting ? ¿Por qué? ¿Cómo podríamos alcanzar el equilibrio entre sesgo y varianza para este caso?

Contesta en el chat de Zoom

CODERHOUSE



Break

¡10 minutos y volvemos!

CODERHOUSE

Validación de modelos

CODERHOUSE

Validación simple

CODERHOUSE

Validación simple



Antes de hablar de la Validación Cruzada, tenemos que entender el concepto de la **Validación Simple**. Este tipo de validación consiste en repartir aleatoriamente las observaciones disponibles en dos grupos, uno se emplea para entrenar al modelo y otro para evaluarlo.

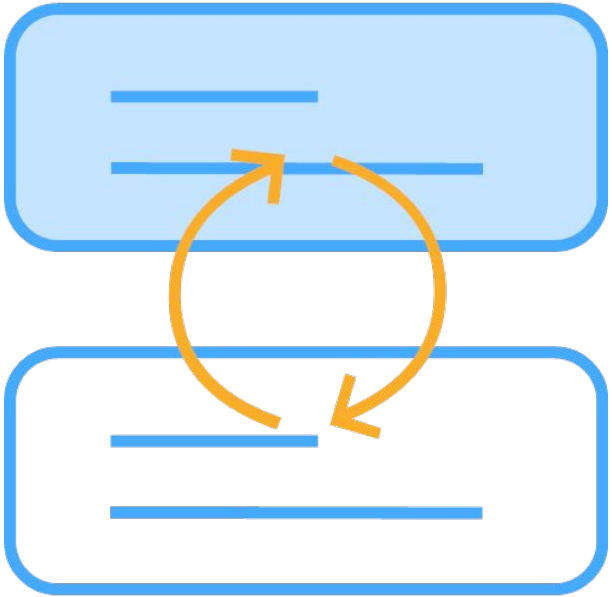


CODERHOUSE

Ventajas de Validación simple



1. Permite una validación rápida de las predicciones de los modelos utilizados
2. No requiere de mucho costo computacional y de tiempo para su ejecución
3. Es sencillo de implementar en diversos lenguajes de programación

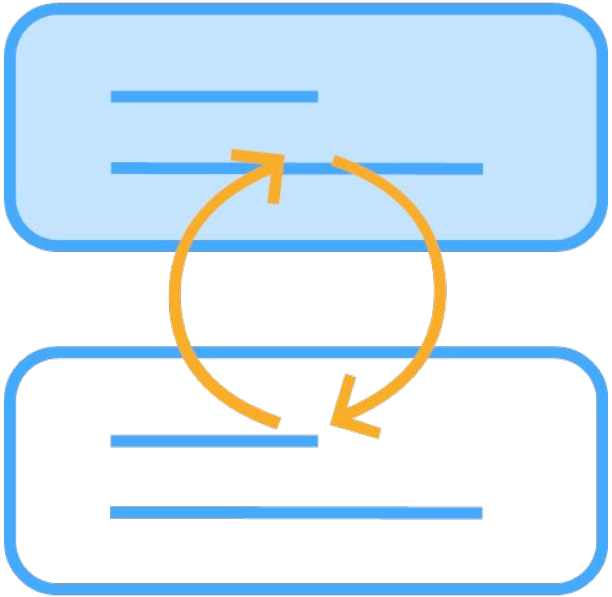


CODERHOUSE

Desventajas de Validación simple



1. La estimación del error es altamente variable, dependiendo de qué observaciones se incluyan como conjunto de entrenamiento y cuáles como conjunto de validación (**problema de varianza**).

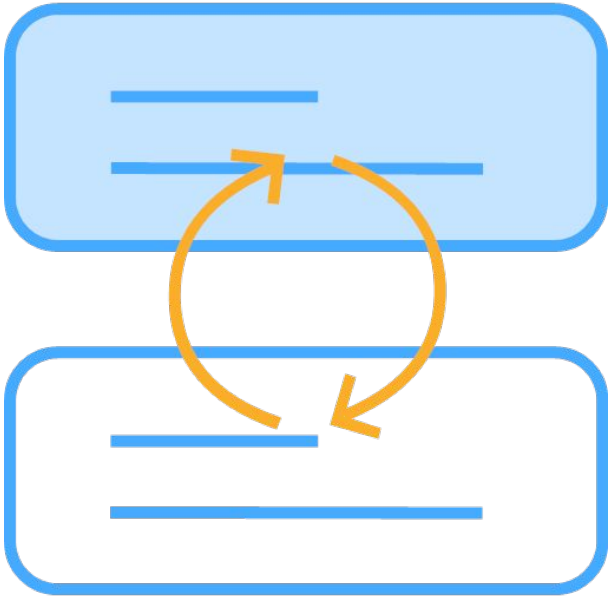


CODERHOUSE

Desventajas de Validación simple



2. Al excluir parte de las observaciones disponibles como datos de entrenamiento (generalmente el 30%), se dispone de menos información con la que entrenar el modelo y, por lo tanto, se reduce su capacidad (**problema de bias**).



CODERHOUSE

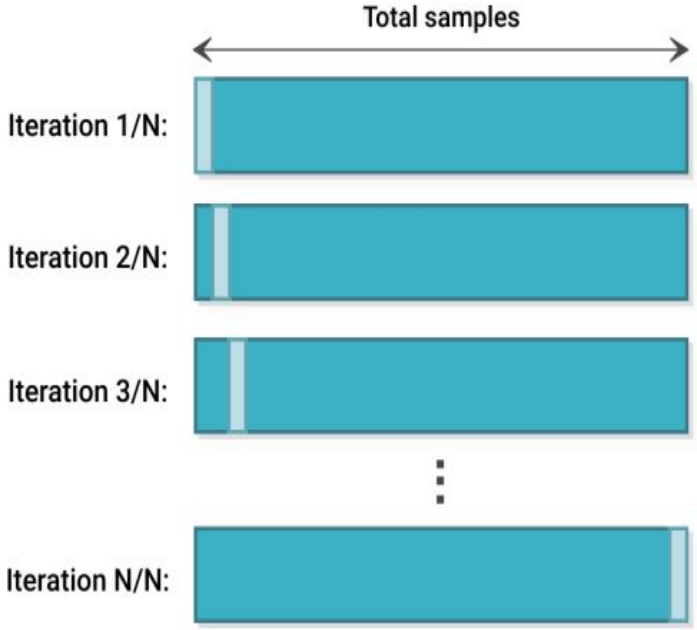
LOOCV

CODERHOUSE

Leave One Out Cross-Validation



Este método es de tipo iterativo y se inicia empleando como conjunto de entrenamiento **todas las observaciones disponibles excepto una, que se excluye para emplearla como validación**.



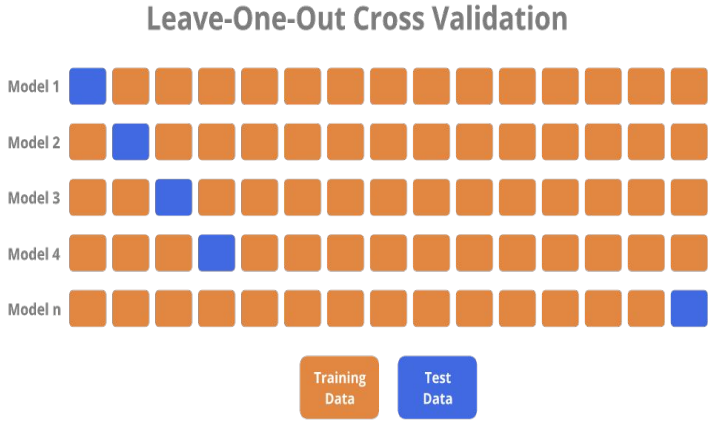
CODERHOUSE

Leave One Out Cross-Validation



Es un caso especial de validación cruzada donde el número de folds es igual al número de instancias en el conjunto de datos.

Si se emplea una única observación para calcular el error, este varía mucho dependiendo de qué observación se haya seleccionado.

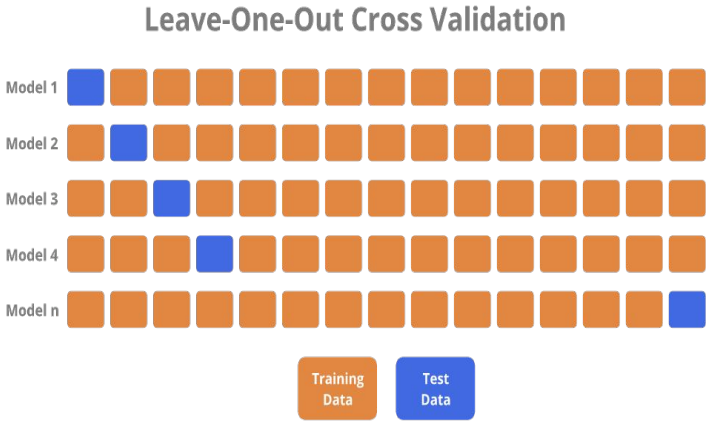


CODERHOUSE

Leave One Out Cross-Validation



Para evitarlo, **el proceso se repite tantas veces como observaciones disponibles, excluyendo en cada iteración una observación distinta,** ajustando el modelo con el resto y calculando el error con dicha observación.

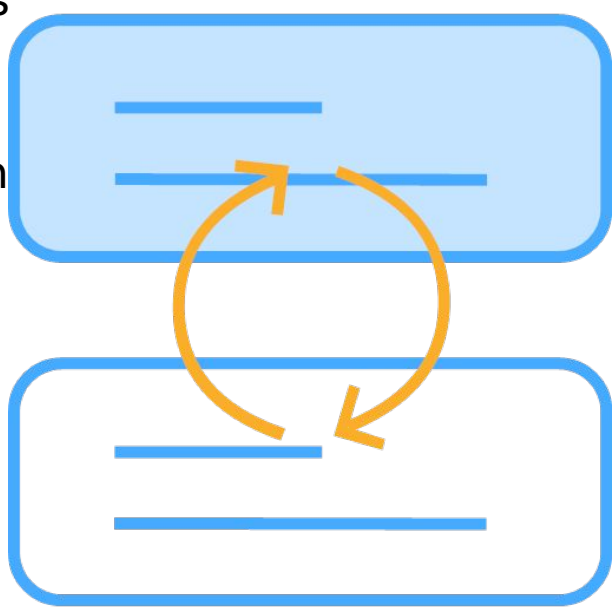


CODERHOUSE

Ventajas de LOOCV



1. No hay aleatoriedad en el uso de algunas observaciones para el entrenamiento frente al conjunto de validación como en el método común
2. Menos sesgo en general para el modelo, ya que el conjunto de entrenamiento tiene un tamaño n-1.

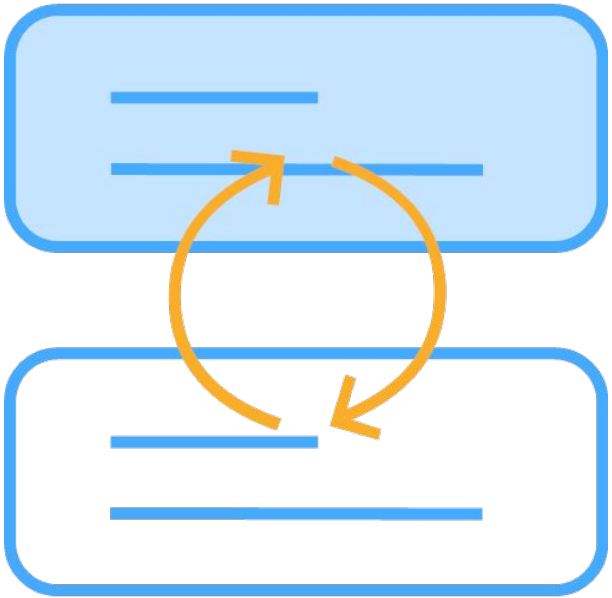


CODERHOUSE

Desventajas de LOOCV



1. Aunque el error de prueba no tiene mucho sesgo, tiene una alta variabilidad ya que solo se usó un conjunto de validación de observación para la predicción.
2. Computacionalmente costoso (tiempo y energía), especialmente si el conjunto de datos es grande, ya que requiere ajustar el modelo n veces



CODERHOUSE

PARA RECORDAR

LOOCV es un método de validación muy extendido ya que puede aplicarse para evaluar cualquier tipo de modelo. Sin embargo, también suele aplicarse el método **K-Fold Cross-Validation**

CODERHOUSE



Ejemplo en vivo

A continuación analizaremos un ejemplo donde pondremos en práctica los conceptos de validación simple y LOOCV para modelos de clasificación y Regresión (Carpeta de clase: Ejemplo 1)

CODERHOUSE

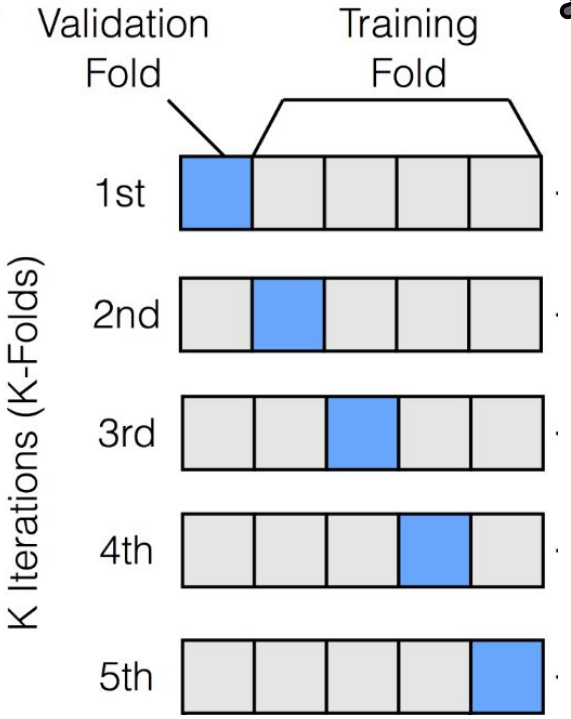
K-Fold Cross Validation

CODERHOUSE

K-Fold Cross Validation

El método K-Fold Cross-Validation es también un proceso iterativo. **Consiste en dividir los datos de forma aleatoria en k grupos de aproximadamente el mismo tamaño, k-1 grupos se emplean para entrenar el modelo y uno de los grupos se emplea como validación.**

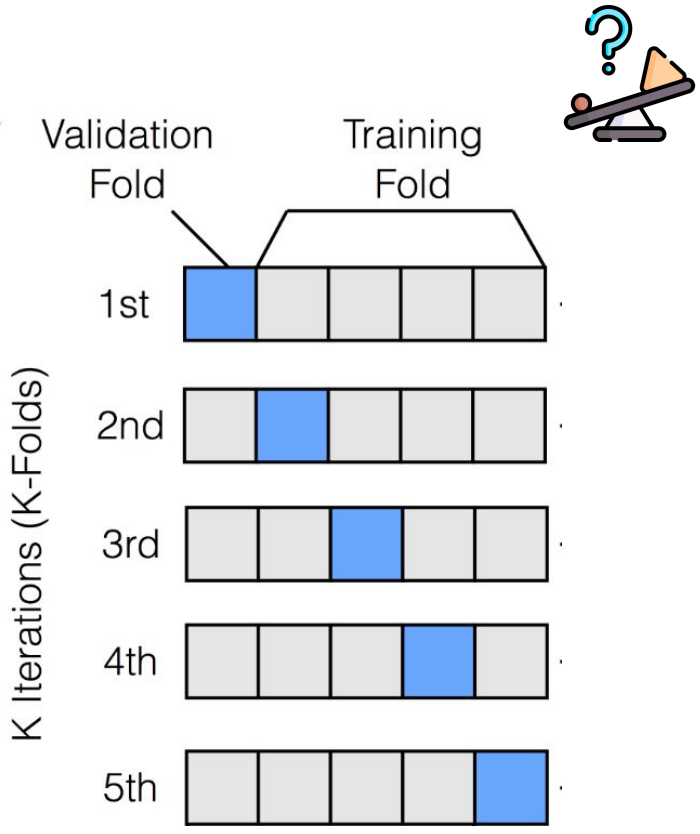




CODERHOUSE

K-Fold Cross Validation

Este proceso se repite k veces utilizando un grupo distinto como validación en cada iteración. El proceso genera k estimaciones del error cuyo promedio se emplea como estimación final.

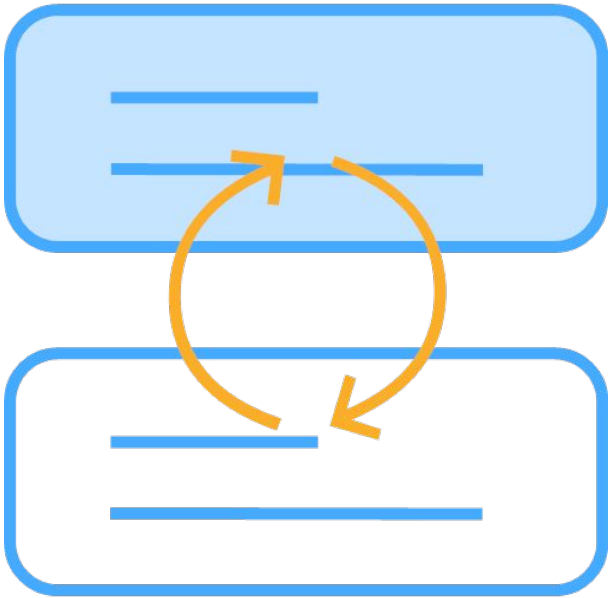


CODERHOUSE

Ventajas de K-Fold Cross Validation



1. Conjunto de validación más grande que en LOOCV, ofrece menos variabilidad en el error de prueba
2. Menos sesgo que en el método de validación simple ya que el conjunto de entrenamiento es más grande
3. Computacionalmente NO es costoso como LOOCV

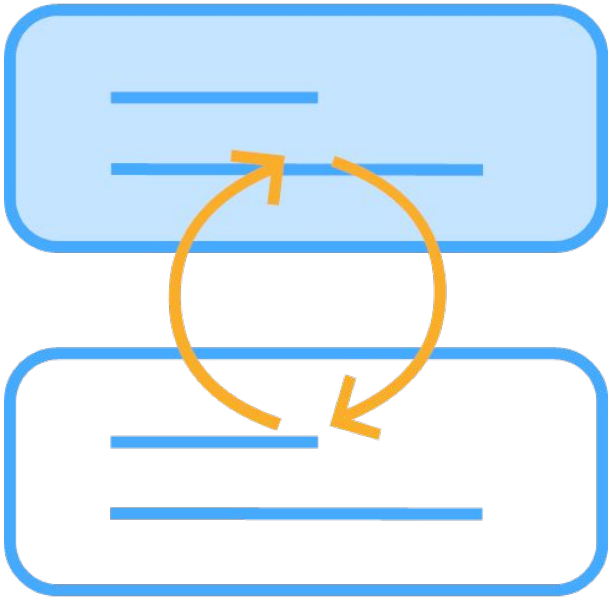


CODERHOUSE

Desventajas de K-Fold Cross Validation



1. Un sesgo algo más alto que LOOCV debido a un conjunto de entrenamiento más pequeño para cada iteración (pero un sesgo más pequeño que el método común de validación).
2. El algoritmo de entrenamiento debe volver a ejecutarse desde cero k veces, lo que significa que se necesitan k veces más cálculos



CODERHOUSE

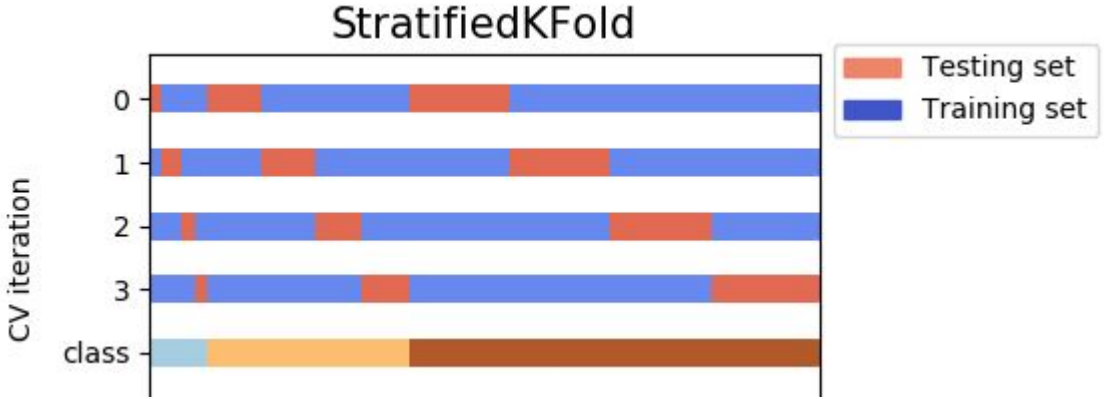
Stratified K-Fold

CODERHOUSE

Stratified K-Fold



Es una variante mejorada de K-fold, que cuando hace los splits (las divisiones) del conjunto de train, tiene en cuenta mantener equilibradas las clases, esto significa que **cada conjunto contiene aproximadamente el mismo porcentaje de muestras de cada clase objetivo que el conjunto completo.**

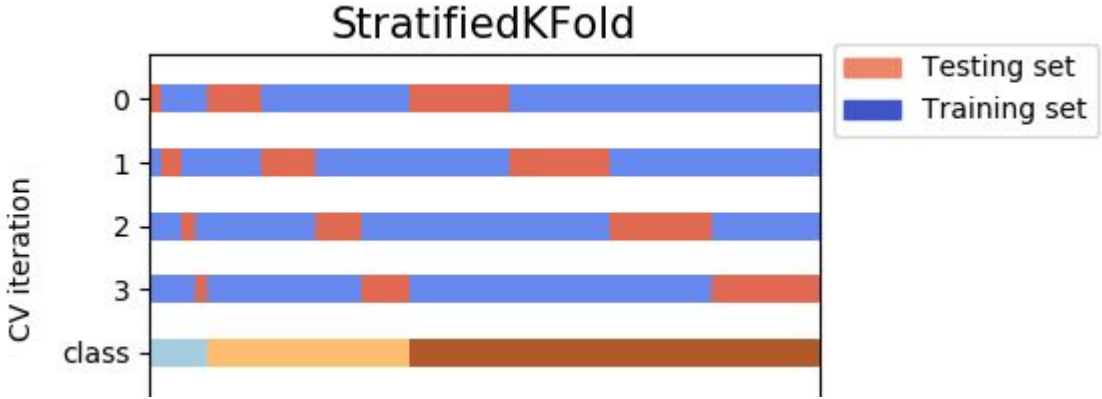


CODERHOUSE

Stratified K-Fold



Esto es muy útil, en el siguiente caso por ejemplo: Tenemos que clasificar nuestro target en “SI/NO” entonces si una de las iteraciones del K-fold normal tuviera muestras con etiquetas sólo “SI” el modelo no podría aprender a generalizar y aprenderá para cualquier input a responder “SI”.

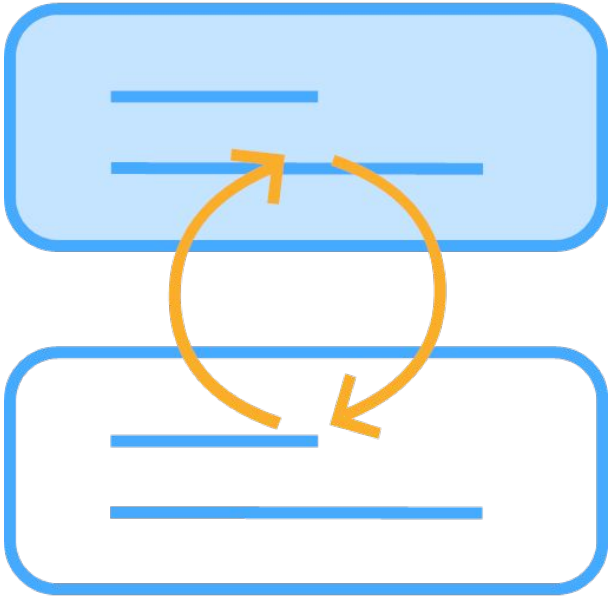


CODERHOUSE

Ventajas de K-Fold Cross Validation



1. 'Valida' el rendimiento de su modelo en múltiples folds de sus datos.
2. Puede equilibrar las clases si se trata de un conjunto de datos desequilibrado.
3. Brinda una respuesta más estable sobre cómo se desempeña su modelo en diversos conjuntos de datos

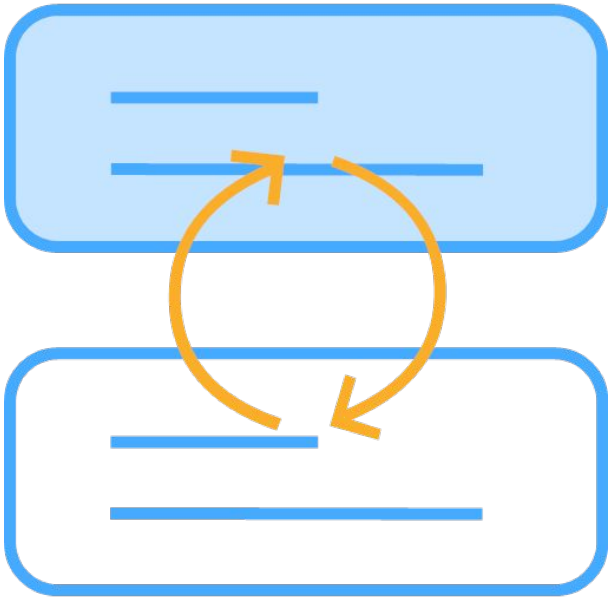


CODERHOUSE

Desventajas de K-Fold Cross Validation



1. K-fold realmente no funciona bien con datos secuenciales (e.g una serie de tiempo de algún tipo).
2. Si solo confía en el puntaje agregado final de K-fold para su modelo se pierde mucha información sobre el rendimiento de los modelos que se puede considerar como un riesgo



CODERHOUSE



Ejemplo en vivo

A continuación analizaremos un ejemplo donde pondremos en práctica las técnicas de Cross validation y Stratified K-Fold para modelos de clasificación y Regresión. (Carpeta de clase: Ejemplo 2 y 3)

CODERHOUSE

Glosario

Bias: Es una forma de cuantificar el error de un modelo y se define como la diferencia entre el valor esperado del estimador

Varianza: Es una medida de dispersión pero se utiliza como estimador de cuánto varía la predicción según los datos que utilicemos para el entrenamiento.

Bias Variance Tradeoff: Consideración de diseño al entrenar modelos de aprendizaje automático. Es un paradigma a la hora de aplicar modelos de Machine Learning

Overfitting: Ocurre cuando el modelo solo funciona correctamente en los datos de train pero no es capaz de generalizar lo aprendido

LOOCV: Este método es de tipo iterativo y se inicia empleando como conjunto de entrenamiento todas las observaciones disponibles excepto una, que se excluye para emplearla como validación.

Stratified K Fold: Es una variante mejorada de K-fold, que cuando hace los splits (las divisiones) del conjunto de train, tiene en cuenta mantener equilibradas las clases

CODERHOUSE

¿Preguntas?

CODERHOUSE

Opina y valora esta clase

CODERHOUSE

Resumen
de la clase hoy

- ✓ Bias vs Variance Tradeoff
- ✓ Validación simple
- ✓ LOOCV
- ✓ K-Fold Cross Validation
- ✓ Stratified K-Fold

CODERHOUSE

Muchas gracias.

CODERHOUSE

#DemocratizandoLaEducación

CODERHOUSE